MODUL PRAKTIKUM: OBJECT-ORIENTED PROGRAMMING
MINGGU 7: Special Classes & Static Members

Waktu Pengerjaan: 100 menit (Lab Session)

Kata-kata pembuka:
"Hari ini kita belajar bahwa Kotlin sudah move on dan membuang masa lalu yang bersifat static. Di dunia nyata, kamu juga tidak bisa terus-terusan mengandalkan global state yang menempel di mana-mana dan sulit dikontrol. Terkadang, kamu harus berani memposisikan dirimu sebagai Singleton—mandiri, unik, dan fokus pada satu tujuan tanpa perlu menduplikasi diri menjadi orang lain. Selain itu, belajarlah dari Data Class: jangan pernah menilai seseorang hanya dari 'alamat' atau asal-usul tempatnya berada, tapi nilailah dari value dan karakter di dalamnya (Structural Equality). Siapkan mental dan jarimu, karena hari ini kita akan membuktikan bahwa untuk menjadi sesuatu yang Special, kamu harus melewati banyak trial and error sebelum akhirnya layak di-commit."

A. ATURAN PROJECT (REMINDER)

Ingat kebijakan Single Repository dari minggu lalu. Jangan buat project baru!

1. Buka project `IF433_Lab_Works` yang sudah dibuat minggu lalu.
2. Di dalam folder `src`, buat package baru `oop_<nim>_<nama>.week07`.
3. Pastikan semua kode hari ini masuk ke package `week07`.

B. LATIHAN TERBIMBING 1: Singleton & Companion Object (Goodbye Static)

Tujuan: Memahami cara kerja `object` sebagai Singleton dan `companion object` sebagai pengganti `static`.

LANGKAH 1: Membuat Singleton dengan `object`

Singleton memastikan hanya ada SATU instansiasi class di seluruh aplikasi, sangat berguna untuk koneksi database.

1. Buat file `DatabaseManager.kt`.
2. Ketik kode berikut. Perhatikan kita menggunakan keyword `object`, bukan `class`.

```kotlin
package oop_001_johnthor.week07

object DatabaseManager {
    var connectionStatus: String = "Disconnected"

    fun connect() {
        connectionStatus = "Connected to Server"
        println("Database is ready.")
    }
}
```

CHECKPOINT 1: *Commit dengan pesan:*

```
"week07: create singleton DatabaseManager"
```

LANGKAH 2: Trial & Error (Private Constructor)

Bagaimana jika kita butuh fungsi statis di dalam kelas yang *bisa* di-instansiasi?

1. Buat file `NetworkClient.kt`.
2. Buat class dengan *private constructor* agar tidak bisa dipanggil sembarangan secara langsung.

```kotlin
package oop_001_johnthor.week07

class NetworkClient private constructor(val url: String) {
    fun connect() {
        println("Connecting to $url...")
    }
}
```

3. Buat file `Main.kt`. Coba instansiasi: `val client = NetworkClient("https://api.umn.ac.id")`. Kamu akan melihat garis merah (Compiler Error) karena konstruktornya private.

CHECKPOINT 2: Commit kode yang masih error di `Main.kt` ini dengan pesan:

```
"week07: (trial) trigger private constructor error on NetworkClient"
```

LANGKAH 3: Resolusi dengan Companion Object (Factory Pattern)

Kembali ke `NetworkClient.kt`. Tambahkan `companion object` yang bertindak sebagai pabrik (Factory) penyedia instansiasi.

```
class NetworkClient private constructor(val url: String) {

    companion object {
        const val BASE_URL = "https://api.umn.ac.id" // Shared constant

        fun createClient(): NetworkClient {
            println("Membangun NetworkClient dengan BASE_URL: $BASE_URL")
            return NetworkClient(BASE_URL)
        }
    }

    fun connect() {
        println("Connecting to $url...")
    }
}
```

CHECKPOINT 3: Commit perbaikan ini dengan pesan:

```
"week07: implement companion object as factory in NetworkClient"
```

LANGKAH 4: Testing di Main

Hapus kode error di `Main.kt` sebelumnya, lalu ganti menjadi:

```
package oop_001_johnthor.week07

fun main() {
    println("=== TEST SINGLETON ===")
    println("Status: ${DatabaseManager.connectionStatus}")
    DatabaseManager.connect()

    println("\n=== TEST COMPANION OBJECT ===")
    val client = NetworkClient.createClient() // Instansiasi lewat Factory
    client.connect()
}
```

CHECKPOINT 4: Commit dengan pesan:

```
"week07: test Singleton and Companion Object in main"
```

C. LATIHAN TERBIMBING 2: Data Class (The Boilerplate Killer)

Tujuan: Membuktikan bahwa `data class` otomatis membuatkan fungsi `equals()`, `hashCode()`, `toString()`, dan `copy()`.

LANGKAH 1: Kelemahan Regular Class (Trial)

1. Buat file `UserModels.kt`.
2. Tambahkan class biasa:

```
package oop_001_johnthor.week07

class RegularUser(val name: String, val age: Int)
```

3. Di `Main.kt`, tambahkan pengujian ini. Kamu akan melihat bahwa mencetak objek akan menghasilkan alamat memori yang tidak terbaca, dan perbandingan `==` akan mengembalikan nilai `false`.

```
println("\n=== TEST REGULAR CLASS ===")
val reg1 = RegularUser("Alice", 22)
val reg2 = RegularUser("Alice", 22)
println(reg1) // Akan mencetak memori hash
println("Sama? ${reg1 == reg2}") // False
```

CHECKPOINT 5 (COMMIT KE-4): *Commit* dengan pesan:

```
"week07: (trial) demonstrate regular class equality failure"
```

LANGKAH 2: Menggunakan Data Class

1. Di `UserModels.kt`, tambahkan `data class`:

```
data class DataUser(val name: String, val age: Int)
```

2. Di `Main.kt`, tambahkan:

```
println("\n=== TEST DATA CLASS ===")
val data1 = DataUser("Alice", 22)
val data2 = DataUser("Alice", 22)
println(data1) // Otomatis readable format
println("Sama? ${data1 == data2}") // True (Structural Equality)
```

CHECKPOINT 6: *Commit* dengan pesan:

```
"week07: implement data class for structural equality"
```

LANGKAH 3: Immutability & Destructuring

Gunakan fungsi bawaan `copy()` untuk memanipulasi data tanpa merusak objek asli (Immutability). Serta fitur *destructuring* untuk memecah objek. Tambahkan di `Main.kt`:

```
val data3 = data1.copy(age = 23)
println("Hasil Copy: $data3")

val (userName, userAge) = data1 // Destructuring Declaration
println("Destructured: $userName berumur $userAge")
```

CHECKPOINT 7: *Commit* dengan pesan:

```
"week07: implement copy and destructuring on data class"
```

D. LATIHAN TERBIMBING 3: Enum & Sealed Classes

Tujuan: Mengelola *State* (status). Enum untuk konstanta sederhana, Sealed Class untuk state dinamis yang membawa *payload* data.

LANGKAH 1: Membuat Enum Class

Enum memastikan data hanya berisi nilai yang sudah didefinisikan secara konstan.

1. Buat file `SystemStates.kt`.
2. Ketik:

```
package oop_001_johnthor.week07

enum class AppState {
    STARTING, RUNNING, STOPPED
}
```

CHECKPOINT 8: *Commit* dengan pesan:

```
"week07: create AppState enum"
```

LANGKAH 2: Membuat Sealed Class

Sealed class adalah hirarki tertutup dimana setiap *child* bisa memiliki *state* dan *property* yang benar-benar berbeda.

Di file `SystemStates.kt`, tambahkan:

```
sealed class ApiResponse {
    data class Success(val data: String) : ApiResponse()
    data class Error(val message: String) : ApiResponse()
    object Loading : ApiResponse()
}
```

CHECKPOINT 9: *Commit* dengan pesan:

```
"week07: create ApiResponse sealed class"
```

LANGKAH 3: Exhaustive `when` Check (Trial & Error)

1. Di `Main.kt`, coba gunakan `when` untuk mengecek `ApiResponse`, **tapi lupakan state `Loading`**:

```
println("\n=== TEST SEALED CLASS ===")
val response: ApiResponse = ApiResponse.Success("Data berhasil ditarik!")

// ERROR: 'when' expression must be exhaustive
val uiMessage = when(response) {
    is ApiResponse.Success -> "Tampilkan: ${response.data}"
    is ApiResponse.Error -> "Munculkan alert: ${response.message}"
}
```

CHECKPOINT 10: *Commit kode error ini dengan pesan:*

```
"week07: (trial) trigger non-exhaustive when compiler error"
```

LANGKAH 4: Resolusi Exhaustive Check

Tambahkan `ApiResponse.Loading -> "Tampilkan Spinner"` ke dalam blok `when` agar compiler memvalidasinya. Cetak `uiMessage` ke konsol.

CHECKPOINT 11: *Commit dengan pesan:*

```
"week07: resolve sealed class exhaustive check"
```

E. TUGAS MANDIRI: Sistem Core Game RPG Sederhana

Skenario: Kamu ditugaskan untuk membangun *core engine* dari sebuah game RPG (*Role-Playing Game*) sederhana. Sistem ini harus efisien dan aman: hanya boleh ada satu manajer utama (Singleton), tingkat kelangkaan item yang sudah pasti (Enum), status pertarungan yang dinamis dan terstruktur (Sealed Class), serta sistem pembuatan senjata yang terenkapsulasi (Companion Object Factory).

Langkah-langkah Tugas:

1. **Singleton Engine (Sistem Utama Game):**
   - Buat file `GameManager.kt`.
   - Buat `object GameManager`.
   - Miliki properti `var isGameRunning: Boolean = false`.
   - Buat method `startGame()`. Jika game sudah berjalan, cetak "Game sudah berjalan! Mencegah instansiasi ganda." Jika belum, ubah ke `true` dan cetak "Memulai Game Engine..."

CHECKPOINT 12: *Commit* dengan pesan:

```
"week07: (task) create GameManager singleton"
```

2. **Enum Tingkat Kelangkaan (Rarity):**
   - Buat file `GameModels.kt`.
   - Buat `enum class ItemRarity` dengan nilai: `COMMON`, `UNCOMMON`, `RARE`, `EPIC`, `LEGENDARY`.
   - Berikan masing-masing enum sebuah properti `val dropChance: Int`. (Misal: `COMMON` -> 70, `LEGENDARY` -> 1).

CHECKPOINT 13: Commit dengan pesan:

```
"week07: (task) create ItemRarity enum"
```

3. **Data Class Item Game:**
   - Masih di `GameModels.kt`, buat `data class GameItem`.
   - Konstruktor: `val name: String`, `val damage: Int`, `val rarity: ItemRarity`.
   - Kelas ini murni penyimpan data *immutable* (gunakan `val` untuk semua properti).

CHECKPOINT 14: Commit dengan pesan:

```
"week07: (task) create GameItem data class"
```

4. **Companion Object sebagai Factory (Pabrik Senjata):**
   - Buat file `WeaponForge.kt`.
   - Buat `class Weapon private constructor(val item: GameItem, val durability: Int)`. (Ingat: *private constructor* agar tidak bisa dipanggil sembarangan).
   - Di dalam `companion object`, sediakan dua fungsi: `fun forgeStarterSword(): Weapon` dan `fun forgeEpicSword(): Weapon`.
   - **Aturan Factory:** Fungsi ini mengotomatiskan pembuatan item. `forgeStarterSword` mengembalikan `Weapon` dengan item ("Pedang Kayu Bapuk", damage 5, COMMON) dan durability 50. Sedangkan `forgeEpicSword` mengembalikan senjata tipe EPIC dengan damage tinggi.

CHECKPOINT 15: Commit dengan pesan:

```
"week07: (task) implement factory pattern in WeaponForge"
```

5. Sealed Class Event Pertarungan (Dynamic Payloads):
   - Buat file `BattleEvent.kt`.
   - Buat `sealed class BattleState`.
   - Buat turunan dari kelas tersebut:
     1. `data class MonsterEncounter(val monsterName: String) : BattleState()`
     2. `data class LootDropped(val item: GameItem) : BattleState()`
     3. `data class GameOver(val reason: String) : BattleState()`
     4. `object SafeZone : BattleState()` (Tanpa payload/data tambahan, hanya penanda status aman).

CHECKPOINT 16: Commit dengan pesan:

```
"week07: (task) create BattleState sealed class hierarchy"
```

6. Logika Eksekutor Event:
   - Buat file `GameExecutor.kt`. Buat fungsi (bukan di dalam class) `fun processEvent(event: BattleState)`.
   - Gunakan blok `when` (harus *exhaustive*).
   - *Destructure* atau gunakan *Smart Cast* pada payload dari dalam Sealed Class saat melakukan pengecekan, dan cetak instruksi yang sesuai ke konsol. Contoh: Jika mendapat loot, cetak nama item dan *rarity*-nya.

CHECKPOINT 17: Commit dengan pesan:

```
"week07: (task) implement exhaustive processEvent"
```

7. Implementasi & Simulasi di Main:
   - Buka kembali `Main.kt`, simulasikan alur berikut di bagian paling bawah: Panggil `GameManager.startGame()` (Lakukan dua kali untuk membuktikan Singleton bekerja menahan duplikasi).

CHECKPOINT 18: Commit dengan pesan:

```
"week07: (task) test GameManager singleton in main"
```

8. Simulasi Factory & Enum:
   - Cetak nilai *drop chance* dari `ItemRarity.LEGENDARY` ke konsol.
   - Gunakan factory `Weapon.forgeStarterSword()` untuk membuat senjata awal pemain. Cetak detail senjatanya ke konsol.

CHECKPOINT 19: Commit dengan pesan:

```
"week07: (task) simulate rarity and factory instantiation"
```

9. Modifikasi Immutability & Sinyal Event:
   - Ceritanya pemain pergi ke *Blacksmith* (Pandai Besi) untuk *upgrade* senjata. Gunakan fungsi `copy()` bawaan dari `GameItem` milik objek senjata awal tadi untuk membuat pedang baru dengan `damage` yang ditingkatkan (misalnya menjadi 25).
   - Lempar simulasi event berurutan dengan memanggil fungsi `processEvent()`:
     1. Panggil `SafeZone`
     2. Panggil `MonsterEncounter("Goblin Nakal")`
     3. Panggil `LootDropped` (masukkan item hasil *copy* tadi sebagai loot-nya)
     4. Panggil `GameOver("Terkena jebakan racun")`.

CHECKPOINT 20: Commit dengan pesan:

```
"week07: (task) test data class copy and sealed class event dispatch"
```

G. PENGUMPULAN

1. **Push ke GitHub:** Pastikan 20 Checkpoint (Commit) di atas sudah terkirim.
   git push
2. **Validasi**: Cek tab "Commits" di repository GitHub Anda. Riwayat pengerjaan *trial and error*, pemecahan Diamond Problem, hingga sistem *Smart Home* harus terlihat.
3. **Submit:** Copy Link Repository ke e-learning UMN (sudah dilakukan di minggu pertama)

Kata-kata penutup:

"Bagaimana rasanya bebas dari jeratan `static` *dan tidak perlu lagi membuang waktu menulis ratusan baris boilerplate secara manual? Kalau hari ini programmu menolak di- compile karena garis merah pada pengecekan Sealed Class, selamat! Kamu baru saja disadarkan bahwa hidup itu menuntut Exhaustive Check—kamu harus punya rencana untuk menghadapi segala kemungkinan yang datang, entah itu Success, Error, ataupun sekadar Loading, tanpa boleh ada skenario yang terlewat. Dan ingat, jika kamu merasa membuat kesalahan hari ini, jangan hancurkan object dirimu. Cukup panggil fungsi copy(), perbaiki atribut yang kurang tepat, dan jadilah versi dirimu yang lebih baik tanpa kehilangan identitas aslimu. Silakan beristirahat, dan pastikan kodemu sudah di-push sebelum laptopnya ditutup."*